

MGSL Trader Screen

React Front-End + N/Cache Map/Reduce

Phase 1: CWP Industriel Inc.

Document Info	
Version	1.1 — Scoped to CWP Industriel Inc.
Status	Draft for Developer Review
Audience	NetSuite SuiteScript Developer
Phase Scope	CWP Industriel Inc. (single subsidiary) only
Cache Strategy	N/cache module — delta refresh every 15 min
Based on	MGSL_TraderScreen_POC_9.html + MCGI_SSU_ * Suitelets

1. Overview & Goals

The MGSL Trader Screen gives traders a real-time view of inventory positions per item and location. The current implementation re-runs expensive NetSuite search queries on every page load, causing slow load times and governance pressure.

Phase 1 goals:

- Scope to CWP Industriel Inc. only. Multi-subsidiary support will be a later phase.
- Replace the server-rendered Suitelet with a React front-end served from a lightweight HTML shell.
- Introduce a Map/Reduce script that pre-computes all inventory data on a schedule and writes the result to the N/cache module, so the React UI reads from a fast in-memory cache rather than running live searches on load.
- Preserve all POC_9 functionality: clickable NS record links, drill-down modals per inventory bucket, UOM conversion, column sorting, drag-to-reorder, filters, and the Create PO / Create SO actions.

2. Architecture

2.1 Component Overview

Component	Script Type	Purpose
Cache Builder	Map/Reduce Script	Runs on a schedule (e.g. hourly). Executes the inventory searches for CWP Industriel Inc., serialises the results to JSON, and writes them to the N/cache module.
Data API	RESTlet (GET + POST)	GET: reads from N/cache and returns JSON to the React app. POST: handles Create PO / Create SO submissions.
Trader Screen Shell	Suitelet	Outputs a minimal HTML page that bootstraps the React app. No server-side data rendering — zero search governance on page load.

2.2 Data Flow

1. Map/Reduce runs on schedule. Computes all inventory buckets and detail rows for every item × location belonging to CWP Industriel Inc. Serialises to JSON and writes to N/cache.
2. Trader opens the Suitelet. The HTML shell loads instantly (no search queries).
3. React mounts and calls GET /restlet with optional filter params. The RESTlet reads from N/cache and returns the JSON payload.
4. React renders the grid client-side. All filtering, sorting, and UOM conversion happen in the browser with no further API calls.
5. Trader clicks a quantity cell. The detail modal opens instantly — data is already in memory from step 3.
6. Trader submits Create PO or Create SO. React POSTs to the RESTlet, which creates the NetSuite record and returns the new record ID and URL.

3. Map/Reduce Cache Script

3.1 Script Identity

Property	Value
NApiVersion	2.1
NScriptType	MapReduceScript
Suggested Script ID	customscript_mcgi_mr_traderscreen_cache
Suggested Deployment ID	customdeploy_mcgi_mr_traderscreen_cache
Schedule	Every 15 minutes. Delta mode by default; full rebuild on first run or cache miss.
Modules required	N/search, N/cache, N/url, N/log, N/runtime

3.2 N/cache Strategy

The N/cache module stores string key-value pairs in a shared memory cache accessible across scripts. It requires no Custom Record creation, no search governance to read, and has built-in TTL expiry.

Cache scope and name

Use `cache.Scope.PUBLIC` so both the Map/Reduce writer and the RESTlet reader share the same cache partition:

```
const myCache = cache.getCache({ name: 'MGSL_TRADERSCREEN_CACHE', scope:
cache.Scope.PUBLIC });
```

Complete key inventory

Key Pattern	Written by	Read by	Contains	TTL
TS_META	Map/Reduce summarize	RESTlet §4.1, §4.2, React	{ cacheVersion, lastUpdated, rowCount, lastRunMode, lastRunTimestamp }	30 min
TS_SUMMARY	Map/Reduce summarize	RESTlet §4.2	JSON array of all summary rows — the full main grid dataset	30 min
TS_DETAIL__{itemId}__{locationId}	Map/Reduce reduce	RESTlet §4.3	JSON object with five detail arrays for that item × location	30 min
TS_LAST_RUN_TIMESTAMP	Map/Reduce summarize	Map/Reduce getInputData	ISO 8601 string — timestamp of the last completed successful run. Used to scope delta searches.	Permanent (no TTL — 86400 s max, refreshed every run)
TS_SUMMARY_CHUNK__{queueIndex}	Map/Reduce reduce	Map/Reduce summarize	Partial summary rows array for one reduce queue. Temporary — deleted in summarize.	30 min

 **Note:** TTL is set to 30 minutes (slightly more than the 15-minute refresh interval). If one cycle fails, the UI continues serving the previous cache. Two consecutive failures would result in a cache miss, which the RESTlet signals to the React app as a `CACHE_MISS 503`.

 **Important:** N/cache values are limited to 500 KB per key. If a single `TS_DETAIL` payload exceeds this limit, split by bucket: `TS_DETAIL_ONHAND__{id}__{loc}`, `TS_DETAIL_COMMITTED__{id}__{loc}`, etc. The developer must measure actual payload sizes during development and choose the appropriate strategy before deploying to production.

3.3 Map/Reduce Phases

The script runs in two modes. On the first run (`TS_LAST_RUN_TIMESTAMP` is absent) or after a cache miss, it performs a Full Rebuild. On subsequent runs it performs an Incremental Delta, processing only item × location pairs whose underlying transaction data changed since the last run.

Mode	When it runs	getInputData input	summarize output
Full Rebuild	First run, or <code>TS_LAST_RUN_TIMESTAMP</code> absent from cache, or manual override via script parameter	All item × location rows from <code>customsearch_suitelet_all_items_search</code>	Replaces <code>TS_SUMMARY</code> entirely. Writes all <code>TS_DETAIL</code> keys.
Incremental Delta	Every subsequent 15-minute cycle	Only item × location pairs touched by transactions modified since <code>TS_LAST_RUN_TIMESTAMP</code>	Upserts changed rows into existing <code>TS_SUMMARY</code> . Overwrites only

Mode	When it runs	getInputData input	summarize output
			affected TS_DETAIL keys.

getInputData

Step 1: Read TS_LAST_RUN_TIMESTAMP from cache. If absent or the script parameter `custscript_ts_force_full_rebuild = true`, set mode = FULL.

Step 2 — Full mode: Load `customsearch_suitelet_all_items_search` with CWP Industriel Inc. subsidiary filter. Use `search.runPaged({ pageSize: 1000 })` to handle large result sets. Return all rows.

Step 2 — Delta mode: Run a lightweight changed-transactions search against the following transaction types: Purchase Order, Sales Order, Item Receipt, Item Fulfillment, Inventory Adjustment, Transfer Order. Apply two filters:

- subsidiary = [CWP Industriel Inc. internal ID]
- lastmodifieddate >= [TS_LAST_RUN_TIMESTAMP value]

Return only the columns: internalid, type, item (join), inventorylocation (join). Extract the unique set of {itemId, locationId} pairs. These are the pairs to reprocess.

Step 3 — Delta mode: For each unique {itemId, locationId} pair, run `customsearch_suitelet_all_items_search` additionally filtered to that specific item and location. Use `runPaged()`. Collect all result rows and pass them to the map phase.

 **Note:** The subsidiary internal ID must never be hardcoded in the script body. Store it as script parameter `custscript_ts_subsidary_id`. Similarly, the force-rebuild flag is script parameter `custscript_ts_force_full_rebuild` (checkbox, default unchecked).

 **Important:** If the delta transaction search returns more than 500 distinct item × location pairs, fall back to a full rebuild for that cycle. This prevents the delta path from consuming more governance than a full rebuild when there has been high transaction volume. The threshold (500) should be a script parameter: `custscript_ts_delta_fallback_threshold`.

map

Emit one key per unique item × location pair observed in the input:

- key = itemInternalId + '___' + locationInternalId
- value = `JSON.stringify({ ...all summary columns from this search result row, _mode: 'FULL' | 'DELTA' })`

reduce

For each key (one item × location pair):

7. Parse the summary data from the map value.
8. Run all five detail searches (on hand, committed, outbound, on order, in transit) filtered to this itemId and locationId. Use `runPaged()` for each.
9. Parse each search result into a typed detail row object per the schemas in §3.5.
10. Assemble the full detail payload: { onHand: [...], committed: [...], outbound: [...], onOrder: [...], inTransit: [...] }.
11. Write to N/cache: key = 'TS_DETAIL___' + itemId + '___' + locationId, value = `JSON.stringify(detailPayload)`, ttl = 1800.
12. Append the summary row object to the queue's chunk array.
13. After all values for this key are processed, write the chunk: key = 'TS_SUMMARY_CHUNK___' + context.key + '___' + queueIndex, value = `JSON.stringify(chunkArray)`, ttl = 1800.

 **Important:** Each reduce queue runs in isolation. Do not attempt to read or merge cross-queue state in reduce. Write one chunk key per queue and assemble only in summarize.

summarize

Full Rebuild mode:

14. Read all TS_SUMMARY_CHUNK__* keys. Merge into a single rows array.
15. Write TS_SUMMARY = JSON.stringify(rows), ttl = 1800.
16. Read previous TS_META. Increment cacheVersion.
17. Write TS_META = JSON.stringify({ cacheVersion, lastUpdated: now, rowCount: rows.length, lastRunMode: 'FULL', lastRunTimestamp: now }), ttl = 1800.
18. Write TS_LAST_RUN_TIMESTAMP = now.toISOString(), ttl = 86400.
19. Delete all TS_SUMMARY_CHUNK__* keys.
20. Log summary: rowCount, duration, governance units consumed.

Incremental Delta mode:

21. Read all TS_SUMMARY_CHUNK__* keys. Merge into a changedRows array (keyed by itemId + '___' + locationId).
22. Read existing TS_SUMMARY from cache. Deserialise into existingRows array.
23. Upsert: for each row in changedRows, find and replace the matching row in existingRows by key. If not found (new item × location combo), append it.
24. Write TS_SUMMARY = JSON.stringify(mergedRows), ttl = 1800.
25. Read previous TS_META. Increment cacheVersion.
26. Write TS_META = JSON.stringify({ cacheVersion, lastUpdated: now, rowCount: mergedRows.length, lastRunMode: 'DELTA', deltaCount: changedRows.length, lastRunTimestamp: now }), ttl = 1800.
27. Write TS_LAST_RUN_TIMESTAMP = now.toISOString(), ttl = 86400.
28. Delete all TS_SUMMARY_CHUNK__* keys.
29. Log summary: deltaCount, duration, governance units consumed.

 **Note:** If TS_SUMMARY is absent during the delta upsert step (cache expired between cycles), abort the upsert and log an error. The next cycle will trigger a full rebuild because TS_LAST_RUN_TIMESTAMP will not be updated.

3.4 Summary Row Schema (written to TS_SUMMARY)

Each element of the TS_SUMMARY array represents one item × location row in the main grid. All URLs are pre-resolved at cache-write time.

Field	Type	NS Source	Notes
internalId	string	internalid summary:MAX	Item NS internal ID
locationId	string	inventorylocation summary:GROUP (value)	Location NS internal ID
locationName	string	inventorylocation summary:GROUP (text)	Human-readable location name
locationUrl	string	url.resolveRecord('location', locationId)	Pre-resolved NS URL
itemCode	string	itemid summary:GROUP	Item code identifier

Field	Type	NS Source	Notes
itemName	string	salesdescription summary:GROUP	Item display description
itemUrl	string	url.resolveRecord(itemType, internalId)	Pre-resolved NS URL
itemType	string	type summary:MAX	'Inventory Item' or 'Assembly'
isReload	boolean	custrecord_is_reload join inventoryLocation	true if location is a Reload
species	string	custitem_species getText	
thickness	string	custitem_mgsl_thickness getText	
width	string	custitem_mgsl_width getText	
length	string	custitem_mgsl_length getText	
grade	string	custitem_grade getText	
finition	string	custitem_finition getText	Finishing / Finition
humidity	string	custitem_humidity getText	
plannage	string	custitem_plannage getText	Planing
etampage	string	custitem_etampage getText	Stamping
autres	string	custitem_autres getText	Other attributes
quantityFBM	number	locationquantityonhand summary:GROUP	Native FBM qty
onHand	number	formula label 'onHand'	Packs on hand (formula)
committed	number	formula label 'committed'	Packs committed (formula)
outbound	number	formula label 'outbound'	Packs outbound (formula)
onOrder	number	formula label 'onOrder'	Packs on order (formula)
inTransit	number	formula label 'inTransit'	Packs in transit (formula)
available	number	formula label 'available'	Packs available (formula)
averageCost	number	locationaveragecost summary:GROUP	Avg cost per pack

3.5 Detail Row Schemas (written to TS_DETAIL__{itemId}__{locationId})

Each detail cache entry is a JSON object with five keys. The React app reads from this when a trader clicks a quantity cell.

```
{ "onHand": [...], "committed": [...], "outbound": [...], "onOrder": [...],  
  "inTransit": [...] }
```

3.5.1 onHand — source: customsearch_mgsl_trader_onhand

Field	NS Source	Notes
docType	type (transaction)	'Item Receipt', 'Inventory Adjustment', etc.

Field	NS Source	Notes
docNum	tranid	Transaction number — displayed as clickable link
docUrl	url.resolveRecord(type, internalid)	Pre-resolved. Type mapped via ITEM_RECORD_TYPE_MAPPING.
receiptDate	trandate	Date string
vendor	mainname getText	Vendor display name — clickable link
vendorUrl	url.resolveRecord('vendor', mainname getValue)	Pre-resolved
lotNo	inventorynumber join inventoryNumber (if applicable)	Lot or serial number
packQty	custcol_mgsl_packqty	Quantity in packs
avgPrice	locationaveragecost	Average cost

3.5.2 committed — source: customsearch_mgsl_trader_committed

Field	NS Source	Notes
docNum	tranid	SO number — clickable link
docUrl	url.resolveRecord('salesorder', internalid)	
customerName	entity getText	Customer display name — clickable link
customerUrl	url.resolveRecord('customer', entity getValue)	
tranDate	trandate	
expectedShipDate	custbody_ship_week	
itemCode	itemid join item	
itemUrl	url.resolveRecord(itemType, item internalid)	
packCommitted	formula label 'Pack Committed'	
openPackQty	formula label 'Open Pack Quantity'	
rate	rate	Price per pack
pricePerPiece	custcol_prixpiece	Price per piece

3.5.3 outbound — source: customsearch_mgsl_trader_outbound

Field	NS Source	Notes
docNum	tranid	SO / Item Fulfillment number — clickable link
docUrl	url.resolveRecord(type, internalid)	Type mapped via ITEM_RECORD_TYPE_MAPPING
customerName	entity getText	Clickable link
customerUrl	url.resolveRecord('customer', entity getValue)	

Field	NS Source	Notes
dueDate	shipdate	
itemCode	itemid join item	
itemUrl	url.resolveRecord(itemType, item internalid)	
packQty	custcol_mgsl_packqty	Total pack quantity
invoicedQty	formula label 'Invoiced Quantity'	
remainingQty	formula label 'Remaining Quantity'	
rate	rate	

3.5.4 onOrder — source: customsearch_mgsl_trader_onorder

Field	NS Source	Notes
docNum	tranid summary:GROUP	PO number — clickable link
docUrl	url.resolveRecord('purchaseorder', internalid summary:GROUP)	
vendorName	entityid join vendor summary:GROUP	Clickable link
vendorUrl	url.resolveRecord('vendor', vendor internalid summary:GROUP)	
shipDate	shipdate summary:GROUP	Expected delivery date
itemCode	itemid join item summary:GROUP	
itemUrl	url.resolveRecord(itemType, item internalid)	
packQty	custcol_mgsl_packqty summary:GROUP	Ordered packs
openQty	formula label 'Open Quantity'	Unfulfilled portion
rate	rate summary:MAX	

3.5.5 inTransit — source: customsearch_mgsl_trader_intransit

Field	NS Source	Notes
docNum	tranid	PO or Transfer Order number — clickable link
docUrl	url.resolveRecord(type, internalid)	
tranDate	trandate	
vendor	mainname getText	Clickable link
vendorUrl	url.resolveRecord('vendor', mainname getValue)	
itemCode	itemid join item	

Field	NS Source	Notes
itemUrl	url.resolveRecord(itemType, item internalid)	
packQty	custcol_mgsl_packqty	
inTransitAdditional	formula label 'In Transit *Additional'	
rate	rate	

3.6 Complete Cache JSON Structure

This section documents the exact JSON shape that the Map/Reduce writes to N/cache and the RESTlet reads back. All field names must match precisely between writer and reader.

3.6.1 TS_META

```
// Key: TS_META | TTL: 1800 s
{
  "cacheVersion":      7,
  "lastUpdated":       "2025-10-14T10:32:11.000Z",    // ISO 8601 UTC
  "rowCount":          312,                          // count of rows in TS_SUMMARY
  "lastRunMode":        "DELTA",                      // "FULL" | "DELTA"
  "deltaCount":         14,                          // rows changed this cycle (DELTA
mode only)
  "lastRunTimestamp":   "2025-10-14T10:32:11.000Z"    // same as lastUpdated – used for
next delta window
}
```

3.6.2 TS_SUMMARY — one row example

```
// Key: TS_SUMMARY | TTL: 1800 s | Value: JSON array of objects like this:
{
  // — Identifiers —
  "internalId":      "1042",
  "locationId":      "3",
  "locationName":    "MTL - Industriel",
  "locationUrl":     "/app/setup/location.nl?id=3",
  "isReload":        false,

  // — Item —
  "itemType":        "inventoryitem",
  "itemCode":        "2x4x16-SPF-KD-STD",
  "itemName":        "2x4 16ft SPF KD STANDARD",
  "itemUrl":         "/app/accounting/items/item.nl?id=1042",

  // — Attributes —
  "species":         "SPF",
  "thickness":        "2\"",
  "width":           "4\"",
  "length":          "16ft",
  "grade":           "Standard",
```

```

"finition":      "KD",
"humidity":      "19%",
"plannage":      "",
"etampage":      "",
"autres":        "",

// — Quantity buckets (Packs) —————
"onHand":        142,
"committed":     38,
"outbound":      12,
"onOrder":       80,
"inTransit":     25,
"available":     92,      // computed: onHand - committed - outbound

// — Pricing —————
"quantityFBM":   28400,  // native FBM qty for UOM conversion
"averageCost":   485.00, // $ per pack

// — React app detail-fetch key —————
"detailKey":     "TS_DETAIL__1042__3"
}

```

 **Note:** The 'available' field is computed by the Map/Reduce formula in the saved search and stored in cache. The React app also recomputes it client-side for the totals footer to account for filtered-row subsets.

3.6.3 TS_DETAIL__{itemId}__{locationId} — full example

```

// Key: TS_DETAIL__1042__3 | TTL: 1800 s
{
  "onHand": [
    {
      "docType":      "Item Receipt",
      "docNum":       "REC-10245",
      "docUrl":       "/app/accounting/transactions/itemrcpt.nl?id=50210",
      "receiptDate":  "2025-09-15",
      "vendor":       "Canfor Ltd.",
      "vendorUrl":    "/app/accounting/vendor.nl?id=88",
      "lotNo":        "LOT-2025-0912",
      "packQty":      40,
      "avgPrice":     482.50
    }
  ],
  "committed": [
    {
      "docNum":       "SO-7821",
      "docUrl":       "/app/accounting/transactions/salesord.nl?id=30881",
      "customerName": "Construction Leblanc Inc.",
      "customerUrl":  "/app/accounting/customer.nl?id=441",
      "tranDate":     "2025-10-01",
      "expectedShipDate": "2025-10-20",
      "itemCode":     "2x4x16-SPF-KD-STD",
      "itemUrl":      "/app/accounting/items/item.nl?id=1042",
      "packCommitted": 10,
      "openPackQty":  10,
      "rate":         510.00,
    }
  ]
}

```

```

    "pricePerPiece": 5.10
  }
],
"outbound": [
  {
    "docNum": "FULFILL-4412",
    "docUrl": "/app/accounting/transactions/itemship.nl?id=44120",
    "customerName": "Matériaux Dupont",
    "customerUrl": "/app/accounting/customer.nl?id=302",
    "dueDate": "2025-10-18",
    "itemCode": "2x4x16-SPF-KD-STD",
    "itemUrl": "/app/accounting/items/item.nl?id=1042",
    "packQty": 12,
    "invoicedQty": 0,
    "remainingQty": 12,
    "rate": 508.00
  }
],
"onOrder": [
  {
    "docNum": "PO-3391",
    "docUrl": "/app/accounting/transactions/purchord.nl?id=33910",
    "vendorName": "Canfor Ltd.",
    "vendorUrl": "/app/accounting/vendor.nl?id=88",
    "shipDate": "2025-11-05",
    "itemCode": "2x4x16-SPF-KD-STD",
    "itemUrl": "/app/accounting/items/item.nl?id=1042",
    "packQty": 80,
    "openQty": 80,
    "rate": 478.00
  }
],
"inTransit": [
  {
    "docNum": "PO-3305",
    "docUrl": "/app/accounting/transactions/purchord.nl?id=33050",
    "tranDate": "2025-10-05",
    "vendor": "Resolute Forest Products",
    "vendorUrl": "/app/accounting/vendor.nl?id=91",
    "itemCode": "2x4x16-SPF-KD-STD",
    "itemUrl": "/app/accounting/items/item.nl?id=1042",
    "packQty": 25,
    "inTransitAdditional": 0,
    "rate": 475.00
  }
]
}

```

3.7 Field Alignment Matrix

This section cross-references every saved search output column against the cache JSON field that stores it, and the React UI element that displays it. Any row marked GAP indicates a missing mapping that must be resolved before development begins.

3.7.1 Main Summary Search → TS_SUMMARY → UI Column

Saved Search Column	Search ID / Formula	Cache JSON Field	UI Column (§6.4)	Status
Item Internal ID	internalid summary:MAX	internalId	— (used internally)	✓ Aligned
Location Internal ID	inventorylocation summary:GROUP value	locationId	— (used internally)	✓ Aligned
Location Name	inventorylocation summary:GROUP text	locationName	Location / Reload (#3)	✓ Aligned
Item Code	itemid summary:GROUP	itemCode	Item Code (#2)	✓ Aligned
Item Description	salesdescription summary:GROUP	itemName	Item Description (#4)	✓ Aligned
Record Type	type summary:MAX	itemType	— (used for URL resolution)	✓ Aligned
Is Reload	custrecord_is_reload join	isReload	Location / Reload (#3) — badge	✓ Aligned
Species	custitem_species getText	species	Species (#5)	✓ Aligned
Thickness	custitem_mgsl_thickness getText	thickness	Thickness (#6)	✓ Aligned
Width	custitem_mgsl_width getText	width	Width (#7)	✓ Aligned
Length	custitem_mgsl_length getText	length	Length (#8)	✓ Aligned
Grade	custitem_grade getText	grade	Grade (#9)	✓ Aligned
Finition	custitem_finition getText	finition	Finition (#10)	✓ Aligned
Humidity	custitem_humidity getText	humidity	Humidity (#11)	✓ Aligned
Plannage	custitem_plannage getText	plannage	Plannage (#12)	✓ Aligned
Étampage	custitem_etampage getText	etampage	Étampage (#13)	✓ Aligned
Autres	custitem_autres getText	autres	Autres (#14)	✓ Aligned
On Hand (formula)	formula label 'onHand'	onHand	On Hand (#15)	✓ Aligned
Committed (formula)	formula label 'committed'	committed	Committed (#16)	✓ Aligned
Outbound (formula)	formula label 'outbound'	outbound	Outbound (#17)	✓ Aligned
On Order (formula)	formula label 'onOrder'	onOrder	On Order (#18)	✓ Aligned
In Transit (formula)	formula label 'inTransit'	inTransit	In Transit (#19)	✓ Aligned
Available (formula)	formula label 'available'	available	Available (#20)	✓ Aligned
Average Cost	locationaveragecost summary:GROUP	averageCost	Avg Price/Pack (#21)	✓ Aligned

Saved Search Column	Search ID / Formula	Cache JSON Field	UI Column (§6.4)	Status
FBM Quantity	locationquantityonhand summary:GROUP	quantityFBM	Used for MBF UOM conversion — not displayed directly	✓ Aligned
Detail Fetch Key	— (assembled in reduce)	detailKey	— (used by React to lazy-load detail)	✓ Aligned
Location URL	url.resolveRecord (reduce)	locationUrl	Location (#3) href	✓ Aligned
Item URL	url.resolveRecord (reduce)	itemUrl	Item Code (#2) href, also all modals	✓ Aligned

⚠ Important: The committed search formula label is spelled 'committed' (one 't') in the existing saved search. The cache JSON field must match this spelling exactly when reading the search result. The React UI label reads 'Committed' (display label is independent). Developer must verify the exact formula label string before going live.

3.7.2 On Hand Search → TS_DETAIL.onHand → On Hand Modal

Saved Search Column	Cache JSON Field	Modal Column	Status
type (transaction)	docType	Doc. Type	✓ Aligned
tranid	docNum	Doc. # (link)	✓ Aligned
url.resolveRecord(type, internalid)	docUrl	href on Doc. #	✓ Aligned
trandate	receiptDate	Receipt Date	✓ Aligned
mainname getText	vendor	Vendor (link)	✓ Aligned
url.resolveRecord('vendor', mainname value)	vendorUrl	href on Vendor	✓ Aligned
inventorynumber join	lotNo	Lot #	⚠ GAP — source field not confirmed. Verify against live search. If items are not lot-tracked, this field will be blank.
custcol_mgsl_packqty	packQty	Quantity (Packs)	✓ Aligned
locationaveragecost	avgPrice	Avg Price	✓ Aligned

3.7.3 Committed Search → TS_DETAIL.committed → Committed Modal

Saved Search Column	Cache JSON Field	Modal Column	Status
tranid	docNum	SO # (link)	✓ Aligned
url.resolveRecord('salesorder', internalid)	docUrl	href on SO #	✓ Aligned
entity getText	customerName	Customer (link)	✓ Aligned
url.resolveRecord('customer', entity value)	customerUrl	href on Customer	✓ Aligned
trandate	tranDate	Trans. Date	✓ Aligned

Saved Search Column	Cache JSON Field	Modal Column	Status
custbody_ship_week	expectedShipDate	Expected Ship Date	✓ Aligned
itemid join item	itemCode	Item Code (link)	✓ Aligned
url.resolveRecord(itemType, item internalid)	itemUrl	href on Item Code	✓ Aligned
formula label 'Pack Committed'	packCommitted	Pack Committed	✓ Aligned
formula label 'Open Pack Quantity'	openPackQty	Open Pack Qty	✓ Aligned
rate	rate	Price/Pack	✓ Aligned
custcol_prixpiece	pricePerPiece	Price/Piece	✓ Aligned

3.7.4 Outbound Search → TS_DETAIL.outbound → Outbound Modal

Saved Search Column	Cache JSON Field	Modal Column	Status
trandid	docNum	Doc. # (link)	✓ Aligned
url.resolveRecord(type, internalid)	docUrl	href on Doc. #	✓ Aligned
entity getText	customerName	Customer (link)	✓ Aligned
url.resolveRecord('customer', entity value)	customerUrl	href on Customer	✓ Aligned
shipdate	dueDate	Ship Date	✓ Aligned
itemid join item	itemCode	Item Code (link)	✓ Aligned
url.resolveRecord(itemType, item internalid)	itemUrl	href on Item Code	✓ Aligned
custcol_mgsl_packqty	packQty	Quantity (Packs)	✓ Aligned
formula label 'Invoiced Quantity'	invoicedQty	Invoiced Qty	✓ Aligned
formula label 'Remaining Quantity'	remainingQty	Remaining Qty	✓ Aligned
rate	rate	Price	✓ Aligned

3.7.5 On Order Search → TS_DETAIL.onOrder → On Order Modal

Saved Search Column	Cache JSON Field	Modal Column	Status
trandid summary:GROUP	docNum	PO # (link)	✓ Aligned
url.resolveRecord('purchaseorder', internalid)	docUrl	href on PO #	✓ Aligned

Saved Search Column	Cache JSON Field	Modal Column	Status
entityid join vendor summary:GROUP	vendorName	Vendor (link)	✓ Aligned
url.resolveRecord('vendor', vendor internalid)	vendorUrl	href on Vendor	✓ Aligned
shipdate summary:GROUP	shipDate	Expected Delivery	✓ Aligned
itemid join item summary:GROUP	itemCode	Item Code (link)	✓ Aligned
url.resolveRecord(itemType, item internalid)	itemUrl	href on Item Code	✓ Aligned
custcol_mgsl_packqty summary:GROUP	packQty	Quantity (Packs)	✓ Aligned
formula label 'Open Quantity'	openQty	Open Qty	✓ Aligned
rate summary:MAX	rate	Price	✓ Aligned

3.7.6 In Transit Search → TS_DETAIL.inTransit → In Transit Modal

Saved Search Column	Cache JSON Field	Modal Column	Status
tranid	docNum	Doc. # (link)	✓ Aligned
url.resolveRecord(type, internalid)	docUrl	href on Doc. #	✓ Aligned
trandate	tranDate	Trans. Date	✓ Aligned
mainname getText	vendor	Vendor (link)	✓ Aligned
url.resolveRecord('vendor', mainname value)	vendorUrl	href on Vendor	✓ Aligned
itemid join item	itemCode	Item Code (link)	✓ Aligned
url.resolveRecord(itemType, item internalid)	itemUrl	href on Item Code	✓ Aligned
custcol_mgsl_packqty	packQty	Quantity (Packs)	✓ Aligned
formula label 'In Transit *Additional'	inTransitAdditional	In Transit Additional	✓ Aligned
rate	rate	Price	✓ Aligned

3.7.7 Unresolved Gaps & Open Questions

ID	Gap / Question	Affects	Action Required
G - 0 1	Lot number source is not confirmed. The current On Hand schema references inventorynumber join, but not all inventory items may be lot-tracked. If items are not serialised, this field will always be blank.	TS_DETAIL.onHand.lotNo, On Hand modal Lot # column	Developer: test the On Hand saved search against live data. Confirm whether lot numbers appear. If not, remove the field from the schema or replace with the correct source.

ID	Gap / Question	Affects	Action Required
G-02	The committed search formula label is spelled 'committed' (single 't') in the existing Suitelet code. This must be the exact string used to read the formula result column.	TS_SUMMARY.committed, TS_DETAIL.committed.packCommitte d	Developer: open customsearch_mgsl_trader_committed in NS and confirm the exact formula label string before writing the cache parser.
G-03	The 'In Transit *Additional' formula label contains an asterisk. Confirm the exact label string (including any spaces or special characters) in customsearch_mgsl_trader_intransit.	TS_DETAIL.inTransit.inTransitAdditional	Developer: inspect the saved search formula column label directly.
G-04	The averageCost field in TS_SUMMARY comes from locationaveragecost. The On Hand modal also shows an avgPrice field per transaction line. These two values may differ. Confirm which source each UI element should use.	TS_SUMMARY.averageCost (Avg Price/Pack column), TS_DETAIL.onHand.avgPrice	Business: confirm whether the main grid Avg Price/Pack should show the NS location average cost, or the average of receipt prices from the On Hand modal rows.
G-05	UOM conversion factor for MBF is not yet defined. The quantityFBM field is cached but the Packs-to-MBF multiplier is unknown.	All quantity columns in UOM=MBF, Avg Price/Pack in UOM=MBF	Business: provide the conversion factor(s) before React development begins.
G-06	The delta change detection searches only transactions by lastmodifieddate. Item master changes (new custitem_ attribute values) will NOT be picked up by the delta path — only a full rebuild captures attribute changes.	All attribute columns in TS_SUMMARY	Business/Developer: decide whether a full rebuild should be forced weekly (or on a separate schedule) to catch item master edits. Implement as an additional scheduled deployment or a script parameter override.

4. RESTlet API

Property	Value
NApiVersion	2.1
NScriptType	Restlet
Suggested Script ID	customscript_mcgi_rl_traderscreen
Modules required	N/cache, N/search, N/record, N/url, N/log

4.1 GET — Check Cache Version (Meta)

A lightweight, fast endpoint the React app uses to check whether newer cached data is available without fetching the full dataset. The Refresh button calls this first.

Request parameters

Param	Type	Required	Description
action	string	Yes	Must be 'meta'

RESTlet logic

- 1. Call myCache.get({ key: 'TS_META' }).
- 2. If null (cache miss), return { available: false, reason: 'CACHE_MISS' }.
- 3. Deserialise and return the meta object. No other cache keys are read.

Response

```
{ "available": true, "cacheVersion": 7, "lastUpdated": "2025-10-14T10:32:11.000Z", "rowCount": 312 }
```

 **Note:** This call reads only the tiny TS_META key. It should complete in under 100 ms and consume < 5 governance units. It is safe to call every few minutes in the background.

4.2 GET — Load Trader Screen Data

Reads from N/cache and returns the filtered summary dataset to the React app. No live search queries are executed.

Request parameters

Param	Type	Description
action	string	Must be 'summary' (or absent — defaults to summary)
location	comma-sep IDs	Filter by inventory location(s)
item	comma-sep IDs	Filter by item internal ID(s)
greaterThanZero	boolean string	When 'true' (default), exclude rows where all buckets = 0
species	comma-sep IDs	
thickness	comma-sep IDs	
width	comma-sep IDs	
length	comma-sep IDs	
grade	comma-sep IDs	
finition	comma-sep IDs	
humidity	comma-sep IDs	
plannage	comma-sep IDs	
etampage	comma-sep IDs	
autres	comma-sep IDs	

RESTlet logic

4. Call `myCache.get({ key: 'TS_SUMMARY' })`. If null (cache miss or expired), return HTTP 503 with { error: 'CACHE_MISS', message: 'Cache is being refreshed. Try again shortly.' }.
5. Deserialise the JSON string into the summary rows array.
6. Apply any filter params (pure JS array filter — no NS search calls).
7. If `greaterThanZero` is true (or absent), filter out rows where `onHand + committed + outbound + onOrder + inTransit === 0`.
8. Compute totals across the filtered rows.
9. Read `TS_META` for `lastUpdated` and `cacheVersion`.
10. For each filtered row, include a `detailKey` field ('TS_DETAIL__{itemId}__{locationId}') so the React app can lazy-load detail rows on demand.
11. Return the response envelope below.

Response envelope

```
{ "rows": [...], "totals": {...}, "meta": { "lastUpdated": "...", "cacheVersion": 7, "rowCount": 312 } }
```

 **Note:** Detail rows are NOT included in the summary response. The React app fetches them lazily via 4.3 when a trader clicks a quantity cell. This keeps the initial payload small.

4.3 GET — Load Detail Rows (lazy)

Called when a trader clicks a quantity cell to open the drill-down modal.

Query Param	Type	Required	Description
action	string	Yes	Must be 'detail'
itemId	string	Yes	Item internal ID
locationId	string	Yes	Location internal ID
bucket	string	No	If provided ('onHand', 'committed', etc.), return only that bucket's array. If absent, return all five.

RESTlet logic

12. Build the cache key: 'TS_DETAIL__' + itemId + '__' + locationId.
13. Call `myCache.get({ key })`. If null, return { error: 'DETAIL_CACHE_MISS' }. The React app should handle this gracefully and show a 'Data unavailable, please wait for next cache refresh' message.
14. Deserialise and return the detail object (or just the requested bucket if param was provided).

4.4 POST — Create Purchase Order or Sales Order

Called when the trader submits the Create PO / Create SO form.

Body Field	Type	Required	Notes
type	'PO' 'SO'	Yes	Determines which record type to create
itemId	string	Yes	Item internal ID
locationId	string	Yes	Location internal ID

Body Field	Type	Required	Notes
partyId	string	Yes	Vendor internal ID (for PO) or Customer internal ID (for SO)
quantity	number	Yes	Quantity in Packs (UI converts from display UOM before sending)
date	string	Yes	ISO 8601 date. Expected delivery date (PO) or expected ship date (SO)
notes	string	No	Memo / special instructions

RESTlet logic

- Validate required fields. Return 400 with field-level errors if invalid.
- Use `record.create()` to create the PO (purchaseorder) or SO (salesorder).
- Set minimum fields: entity (partyId), item line (itemId), quantity, location, expected date (shipdate or custbody equivalent), memo (notes).
- `record.save()`. Return { docId, docNum, docUrl } on success.
- Wrap in try/catch. Return { error, message } on failure.

 **Note:** Exact required fields for PO and SO record creation must be verified against the live account (some custom mandatory fields may exist). Developer should test `record.create()` interactively before finalising the RESTlet.

5. Suitelet Shell

Property	Value
NApiVersion	2.1
NScriptType	Suitelet
Suggested Script ID	customscript_mcgi_ssu_traderscreen_react
Response type	Raw HTML (context.response.write — NOT <code>serverWidget.createForm</code>)
Modules required	N/url, N/runtime, N/log

The Suitelet renders a complete HTML document. It must include:

- IBM Plex Sans and IBM Plex Mono fonts (Google Fonts CDN).
- React 18 and ReactDOM UMD bundles from `cdnjs.cloudflare.com`.
- The compiled React app JS (either hosted as a file cabinet resource or inlined — developer's choice).
- A `<script>` block that defines `window.__NS_CONFIG__` before the app script loads.

`__NS_CONFIG__` object

```

window.__NS_CONFIG__ = {
  restletUrl:
    '/app/site/hosting/restlet.nl?script=customscript_mcgi_rl_traderscreen&deploy=1',
  userId: '<resolved server-side via runtime.getCurrentUser().id>',
  userRole: '<resolved server-side via runtime.getCurrentUser().role>',
  accountId: '<resolved server-side via runtime.accountId>',
  subsidiary: { id: '<CWP IND internal ID>', name: 'CWP Industriel Inc.' }
};

```

✂ **Note:** The `restletUrl` must be resolved server-side using `url.resolveScript({ scriptId, deploymentId, returnUrl: false })` so it includes the correct account-specific domain. Do not hardcode the URL.

6. React Front-End Specification

6.1 Technology

- React 18 (UMD or compiled bundle).
- No external CSS framework. Inline styles using design tokens (§6.2).
- Fonts: IBM Plex Sans (body text), IBM Plex Mono (numeric cells, doc numbers, codes).
- All filter/sort/UOM logic runs client-side on the loaded data — no additional API calls except detail lazy-load and PO/SO creation.

6.2 Design Tokens

Token	Hex	Usage
Navy	#0F2641	Header bg, modal header gradient start
Navy Mid	#1A3D63	Header gradient end, accents
Green	#1E6B47	Available column, On Hand accent
Gold	#C8A035	Sort indicator, totals footer border, Avg Price
Background	#EEF1F6	Page background
Surface	#FFFFFF	Table rows, modal bg
Border	#CBD5E1	All borders
Text	#0D1F33	Primary text
Text Mid	#3D5166	Secondary text
Text Light	#7A8FA3	Hints, sub-labels
Row Hover	#F0F7F4	Table row hover
Row Alt	#F8FAFC	Alternating row shade
Expanded Bg	#F0F5FF	Expanded / expanded modal row

6.3 Page Layout

Header bar

- MGSL wordmark / logo (left).
- Page title: 'Trader Screen — CWP Industriel Inc.' (centre or left — Phase 1 is single-subsiary).
- UOM selector: dropdown with options Packs, MBF — confirm exact options with business (right).
- 'Last updated: [timestamp]' badge — see §6.11 for full state machine.
- 'Refresh' button — see §6.11 for full state machine.
- 'Filters' toggle button. Shows count of active filters as a badge (e.g. 'Filters ③').

Filter panel

- Collapsible panel, collapsed by default. Toggle via a 'Filters' button in the header bar.
- Fields: Location (multi-select), Item (multi-select), Species, Thickness, Width, Length, Grade, Finition, Humidity, Plannage, Éstampage, Autres, and a 'Quantity > 0' toggle (on by default).
- Filter values are sourced from the unique values already present in the loaded summary rows — no additional API call required.
- Applying filters re-filters client-side instantly. Does NOT re-call the RESTlet (data is already loaded).
- Active filters are shown as dismissible chips/tags above the table.

Main data table

- Full-width, full-height scrollable container.
- Sticky column headers (position: sticky, top: 0).
- Sticky totals footer row (position: sticky, bottom: 0).
- Horizontally scrollable when viewport width < total table width.

6.4 CWP Industriel Inc. — Column Set

Phase 1 uses the following fixed column order. Drag-to-reorder is supported for all columns except 'order' (the \$ action column). Column order changes are session-only (not persisted).

#	Column Key	Label	Width	Behaviour
1	order	\$ (action)	36px	Non-sortable, non-draggable. Shows Create PO/SO popover button.
2	itemCode	Item Code	130px	Sortable. Clickable link → item record in NS (new tab). Non-draggable (pinned).
3	location	Location / Reload	165px	Sortable. Clickable link → location record. Also an attribute filter cell.
4	itemName	Item Description	200px	Sortable. Read-only text.
5	species	Species	90px	Sortable. Attribute cell: click to filter by this value.
6	thickness	Thickness	90px	Sortable. Attribute cell.
7	width	Width	90px	Sortable. Attribute cell.
8	length	Length	90px	Sortable. Attribute cell.
9	grade	Grade	90px	Sortable. Attribute cell.
10	finition	Finition	90px	Sortable. Attribute cell.
11	humidity	Humidity	90px	Sortable. Attribute cell.
12	plannage	Plannage	90px	Sortable. Attribute cell.
13	etampage	Éstampage	90px	Sortable. Attribute cell.
14	autres	Autres	90px	Sortable. Attribute cell.
15	onHand	On Hand	95px	Sortable. Right-aligned. Qty cell: click (if > 0) → On Hand modal. Green badge in header.
16	committed	Committed	95px	Sortable. Right-aligned. Qty cell. Orange badge.
17	outbound	Outbound	95px	Sortable. Right-aligned. Qty cell. Pink badge.

#	Column Key	Label	Width	Behaviour
18	onOrder	On Order	95px	Sortable. Right-aligned. Qty cell. Blue badge.
19	inTransit	In Transit	100px	Sortable. Right-aligned. Qty cell. Purple badge.
20	available	Available	100px	Sortable. Right-aligned. Computed client-side. Green-tinted cell background. Not a qty drill-down cell (calculated field).
21	avgPrice	Avg Price/Pack	115px	Sortable. Right-aligned. Gold-tinted cell background. Adjusts for UOM.

6.5 Quantity Cell Behaviours

- A qty cell is clickable only when its value > 0.
- Cells with value 0 display as '—' (em dash) and are not clickable.
- Hovering a clickable qty cell changes its background to a light blue (#EDF4FF) and shows a pointer cursor.
- Hovering an attribute cell changes background to light green (#E8F5EF) and underlines the text.
- Clicking a qty cell triggers a lazy-load GET to the RESTlet for that item × location detail, then opens the Detail Modal. Show a spinner in the cell while loading.

6.6 UOM Conversion

All quantities stored in the cache are in Packs. Conversion is purely client-side.

UOM	Conversion Factor	Notes
Packs	× 1.0 (base unit)	Default
MBF	TBD — confirm with business	Board feet equivalent

 **Note:** Exact conversion factors and the full list of available UOM options for CWP IND must be confirmed with the business before development. Store them as a script parameter or a configurable JSON block — not hardcoded in the React component.

When the UOM changes, all displayed quantities and the Avg Price column update instantly client-side. The totals footer also recalculates. No API call is made on UOM change.

6.7 Detail Drill-Down Modals

Common modal structure

- Backdrop overlay (click outside to close).
- Modal box: max-width 1200px, max-height 88vh, scrollable body.
- Header: bucket icon + label + 'Transaction Detail' + item code + location. Total quantity in current UOM on the right. × close button.
- Sub-header strip: 'Quantities in [UOM]' + row count.
- Table: transaction rows. Alternating row colours. Sticky table header within scroll container.
- Footer row: 'TOTAL' + summed quantity.
- Each row has a \$ button for Create PO/SO in context of that transaction.

On Hand modal — columns

Doc. Type · Doc. # (link) · PO # (link) · Receipt Date · Vendor (link) · Lot # · Quantity · Avg Price

Committed modal — columns

SO # (link) · Expected Ship Date · Customer (link) · Trans. Date · Item Code (link) · Pack Committed · Open Pack Qty · Price/Pack · Price/Piece

Outbound modal — columns

Doc. # (link) · Customer (link) · Ship Date · Item Code (link) · Quantity · Invoiced Qty · Remaining Qty · Price

On Order modal — columns

PO # (link) · Vendor (link) · Expected Delivery · Item Code (link) · Quantity · Open Qty · Price

In Transit modal — columns

Doc. # (link) · Trans. Date · Vendor (link) · Item Code (link) · Quantity · In Transit Additional · Price

6.8 Clickable Links — Complete Reference

Entity	NS Record Type	Where it appears
Item Code	inventoryitem or assemblyitem (from itemType field)	Main table, all modals
Location / Reload	location	Main table
Customer	customer	Committed modal, Outbound modal
Vendor	vendor	On Hand modal, On Order modal, In Transit modal
Sales Order (SO #)	salesorder	Committed modal, Outbound modal
Purchase Order (PO #)	purchaseorder	On Hand modal, On Order modal, In Transit modal
Item Fulfillment	itemfulfillment	Outbound modal (when type = 'ItemShip' / 'ItemFulfillment')
Item Receipt	itemreceipt	On Hand modal (when type = 'ItemRcpt')
Journal Entry (via Linked JE Suitelet)	Redirect via MCGI_SSU_ListLinkedJE.js	Optional: Committed / Outbound modal SO rows

 **Note:** All URLs are pre-resolved by the Map/Reduce script using `url.resolveRecord()` and stored in the cache. The React app uses them directly as href values — no URL construction in the browser.

6.9 Create PO / Create SO

Entry points

- Main table: \$ button in the 'order' column. Opens an OrderPopover with 'Create Purchase Order' and 'Create Sales Order' options.

- Detail modal row: each row also has a \$ button. Pre-fills the form with that row's quantity, vendor/customer, and any existing PO/SO reference.

CreateOrderModal fields

Field	Type	Required	Pre-filled from
Vendor (PO) / Customer (SO)	Select	Yes	Clicked row's vendor/customer if available
Quantity (Packs)	Number	Yes	Clicked row's quantity (converted back to Packs if UOM ≠ Packs)
Expected Delivery / Ship Date	Date picker	Yes	—
Notes	Textarea	No	—

On successful POST: show confirmation with the new document number as a clickable link that opens the NS record in a new tab. On error: show inline error message with the server's error text.

6.10 Loading & Error States

- Initial load: skeleton placeholder rows (shimmering bars) while waiting for the RESTlet GET response.
- Detail load: spinner inside the clicked qty cell while the detail GET is in flight.
- Cache miss (503): banner — 'Cache is refreshing, please try again in a few minutes' with a Retry button.
- Filter produces 0 results: centred empty state —  No results for these filters.
- API error: dismissible error banner at the top with status code and Retry.

6.11 Refresh Button — State Machine

The Refresh button uses a two-step check-then-fetch strategy: it calls the lightweight meta endpoint first (§4.1) to compare cache versions, and only fetches the full dataset if something new is actually available. This avoids unnecessary large payloads when the cache has not changed.

React state variables

State Variable	Type	Initial Value	Purpose
loadedCacheVersion	number null	null	The cacheVersion returned by the last successful full data load.
refreshState	string	'idle'	Controls button appearance and interaction. See states below.
lastChecked	Date null	null	Timestamp of the last meta check (manual or background).
newVersionAvailable	boolean	false	Set to true when a background poll detects a newer cache version.

refreshState values

State	Button label	Button style	Meaning
'idle'	↻ Refresh	Navy outline, enabled	Default. Data is loaded. No pending check or fetch.
'checking'	Checking...	Grey, disabled, spinner	Meta request (§4.1) is in flight.
'up-to-date'	✓ Up to date	Green, disabled, 2 s then → idle	Meta returned same cacheVersion as loadedCacheVersion.
'fetching'	Loading...	Navy filled, disabled, spinner	Full summary request (§4.2) is in flight.
'error'	⚠ Retry	Amber outline, enabled	Either the meta call or the summary call failed.

Manual Refresh — click handler flow

20. Set refreshState = 'checking'. Disable the button.
21. Call GET ?action=meta (§4.1).
22. If the call fails: set refreshState = 'error'. Show dismissible error banner. Stop.
23. If meta.available === false (cache miss): show banner 'Cache is being rebuilt, try again shortly'. Set refreshState = 'idle'. Stop.
24. If meta.cacheVersion === loadedCacheVersion: set refreshState = 'up-to-date'. After 2 seconds, revert to 'idle'. Stop — do not fetch.
25. If meta.cacheVersion > loadedCacheVersion: set refreshState = 'fetching'. Call GET ?action=summary with the current active filters (§4.2).
26. If the summary call fails: set refreshState = 'error'. Show dismissible error banner. Stop.
27. On success: replace the rows and totals in React state. Set loadedCacheVersion = meta.cacheVersion. Clear the in-memory detail cache (all previously lazy-loaded detail rows are stale). Update the Last Updated timestamp. Set refreshState = 'idle'. Show a brief green 'Updated' toast (1.5 s).

 **Note:** The in-memory detail cache is a simple Map keyed by 'itemId_locationId'. When the summary is refreshed, clear it entirely so that detail modals re-fetch from the new cache on next click.

Background polling

In addition to the manual Refresh button, the app polls the meta endpoint silently every 5 minutes (configurable). If a new version is detected, a non-intrusive banner appears at the top of the table:

"  New data available. [Load now] [Dismiss] "

- 'Load now' runs step 6 onwards of the manual refresh flow (skip straight to fetching — version check already done).
- 'Dismiss' closes the banner and resets newVersionAvailable = false. The banner re-appears at the next poll interval if data is still newer.
- If the user is filling out a CreateOrderModal, suppress the banner until the modal is closed.
- Background polling pauses when the browser tab is hidden (Page Visibility API: document.hidden === true) and resumes on focus.

Last Updated badge states

Condition	Badge colour	Text
Data loaded, lastUpdated < 1 hour ago	Grey / subdued	'Last updated: [time]'

Condition	Badge colour	Text
Data loaded, lastUpdated 1–2 hours ago	Amber text, ⚠ icon	'Last updated: [time] — may be stale'
Data loaded, lastUpdated > 2 hours ago	Amber background, ⚠ icon	'Last updated: [time] — refresh recommended'
refreshState = 'checking'	Grey, italic	'Checking for updates...'
refreshState = 'fetching'	Grey, italic	'Loading new data...'
Cache miss on initial load	Red text	'Cache unavailable — retry'

7. Linked Journal Entry Suitelet (No Change)

MCGI_SSU_ListLinkedJE.js is unchanged. It accepts a transaction query parameter (SO internal ID), loads customsearch_mcgi_linked_je with a filter on custcol_je_so_number, and redirects to the NS search results.

The React app may link to it from Committed and Outbound modal rows:

```
/app/site/hosting/scriptlet.nl?script=customscript_mcgi_ssu_listlinkedj&deploy=1&transaction={soInternalId}
```

8. Non-Functional Requirements

8.1 Performance Targets

Operation	Target
Suitelet HTML shell load	< 500 ms
RESTlet GET meta check (§4.1)	< 200 ms
RESTlet GET summary (§4.2, 0–500 rows)	< 1 second
RESTlet GET summary (§4.2, 500–2000 rows)	< 3 seconds
Detail modal open (lazy GET §4.3)	< 1 second
Client-side filter / sort / UOM change	< 100 ms (no API call)
Create PO / SO POST	< 5 seconds

8.2 Governance

- Map/Reduce: Use runPaged() for all five detail searches inside reduce. Target < 10,000 governance units per full refresh cycle for the typical CWP IND catalogue size. Monitor and tune if needed.
- RESTlet GET: 0 search governance (cache read only). Total governance per call should be < 10 units.
- Suitelet shell: 0 governance (HTML write only).

- Map/Reduce concurrency: Set the deployment to use the maximum allowed queue count to parallelise reduce processing.

8.3 N/cache Limits — Developer Must Verify

⚠ Important: N/cache imposes a 500 KB limit per cache value. Before finalising the cache key design, the developer must measure the JSON size of a typical TS_DETAIL payload for a large item. If detail payloads exceed the limit, split by bucket (one key per bucket per item × location) or apply field pruning (remove redundant attribute fields from detail rows that are already present in the summary row).

Limit	Value	Mitigation if exceeded
Max value size per key	500 KB	Split detail payload by bucket (5 keys instead of 1)
Max TTL	24 hours (86400 s)	Not an issue — 90 min TTL used
Cache scope	PUBLIC required for cross-script access	Ensured by design

8.4 Browser Compatibility

- Chrome (latest), Firefox (latest), Edge (latest).
- Minimum viewport: 1280px. Table is horizontally scrollable below this width.

8.5 Security

- All API calls use the existing NetSuite session cookie. No tokens exposed to the browser.
- RESTlet deployment should be role-restricted to Trader roles only.
- Average cost and pricing columns: confirm with business whether all Trader roles should see these, or whether column visibility should vary by role using __NS_CONFIG__.userRole.

9. Migration & Backward Compatibility

- The existing MCGI_SSU_TraderScreen_v2.js and its five sub-Suitelets remain active during UAT.
- The new React Trader Screen is deployed under a separate script ID and a separate menu item so both run in parallel.
- All existing saved searches are reused as-is by the Map/Reduce script. No saved searches are modified.
- Once UAT is signed off for CWP Industriel Inc., the old Suitelet can be retired.

10. Open Questions

#	Question	Owner
1	What are the exact UOM options and conversion factors for CWP IND? (Packs → MBF factor, any others?)	Business
2	What is the internal ID of the CWP Industriel Inc. subsidiary? This must be set as a script parameter before deployment.	Developer

#	Question	Owner
3	Should the 'Autres' and 'Étampage' columns be shown as a single combined cell or as separate columns?	Business
4	Are lot numbers sourced from inventory number records (serialised items) or from a custom field on transaction lines?	Developer
5	For Create PO/SO in Phase 1: should the form create the record fully, or redirect the trader to the standard NS PO/SO form pre-filled?	Business
6	What roles should have access to the new React Trader Screen Suitelet deployment?	Business
7	Should average cost and pricing columns be hidden for any specific roles?	Business
8	After measuring payload sizes: if TS_DETAIL exceeds 500 KB, should the fallback be (a) split by bucket, or (b) omit some fields from detail rows?	Developer
9	Is the 1-hour cache refresh frequency acceptable for Phase 1, or is a shorter interval required?	Business
10	Please confirm the exact saved search IDs for all five detail searches before development begins: customsearch_mgsl_trader_onhand, customsearch_mgsl_trader_committed, customsearch_mgsl_trader_intransit, customsearch_mgsl_trader_outbound, customsearch_mgsl_trader_onorder.	Developer

End of Document